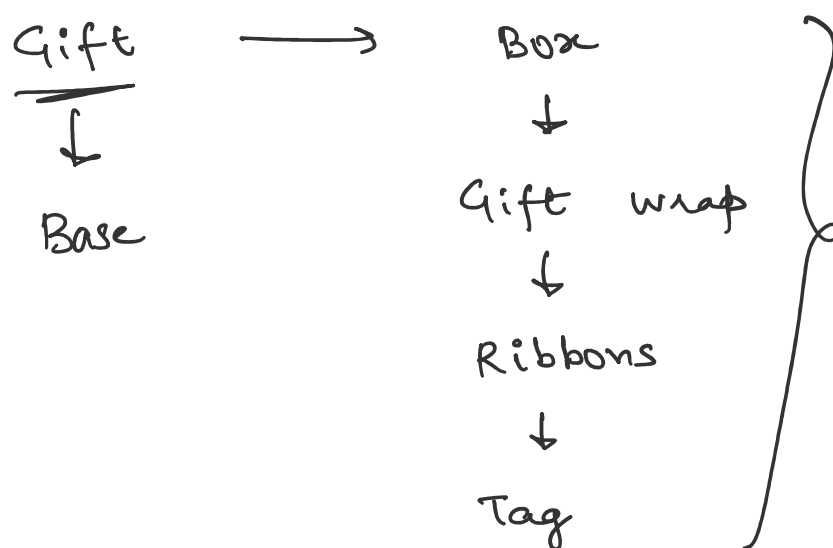
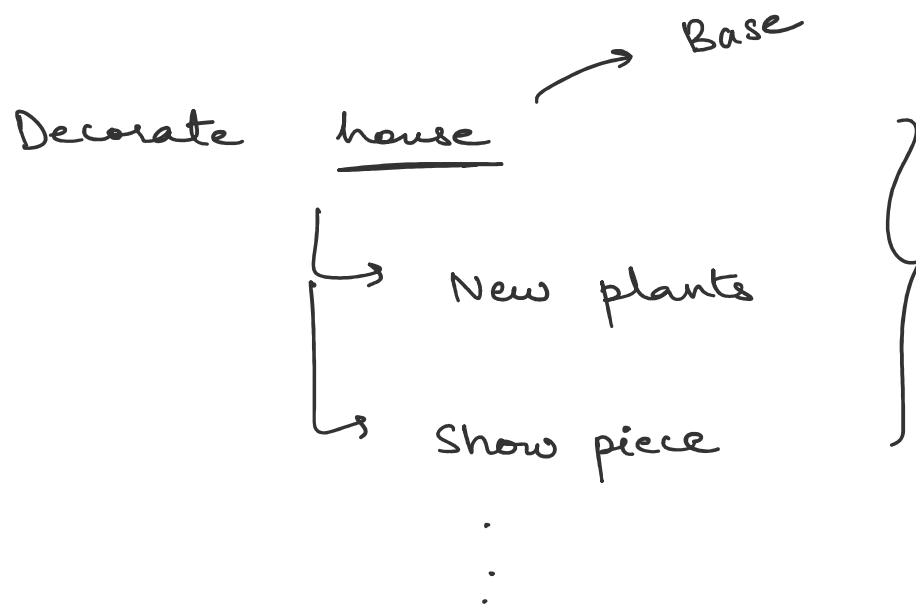


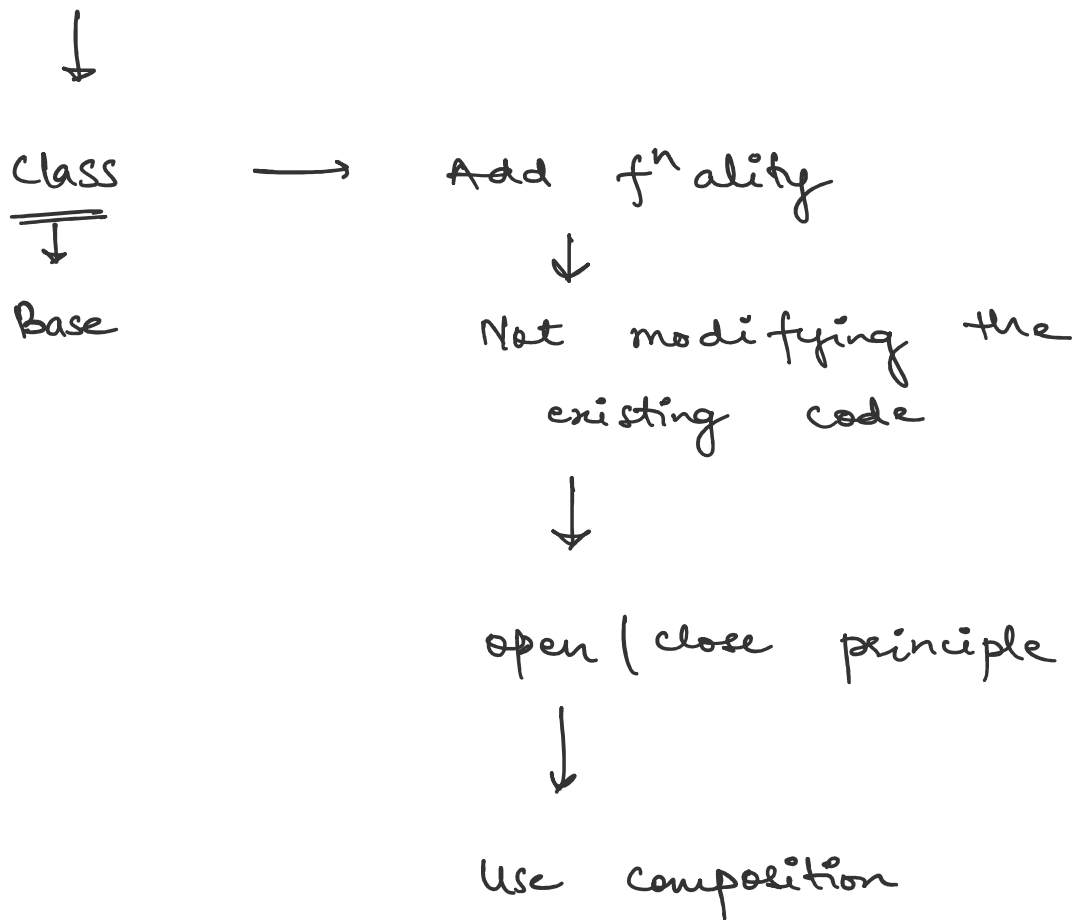
Class 13

Saturday, July 4, 2026 5:39 PM

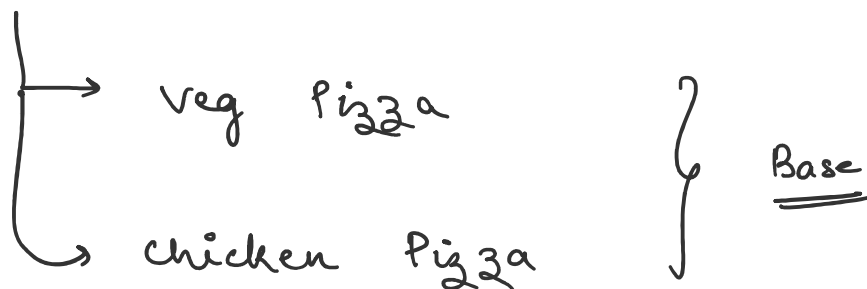
Decorator DP



Decorator DP



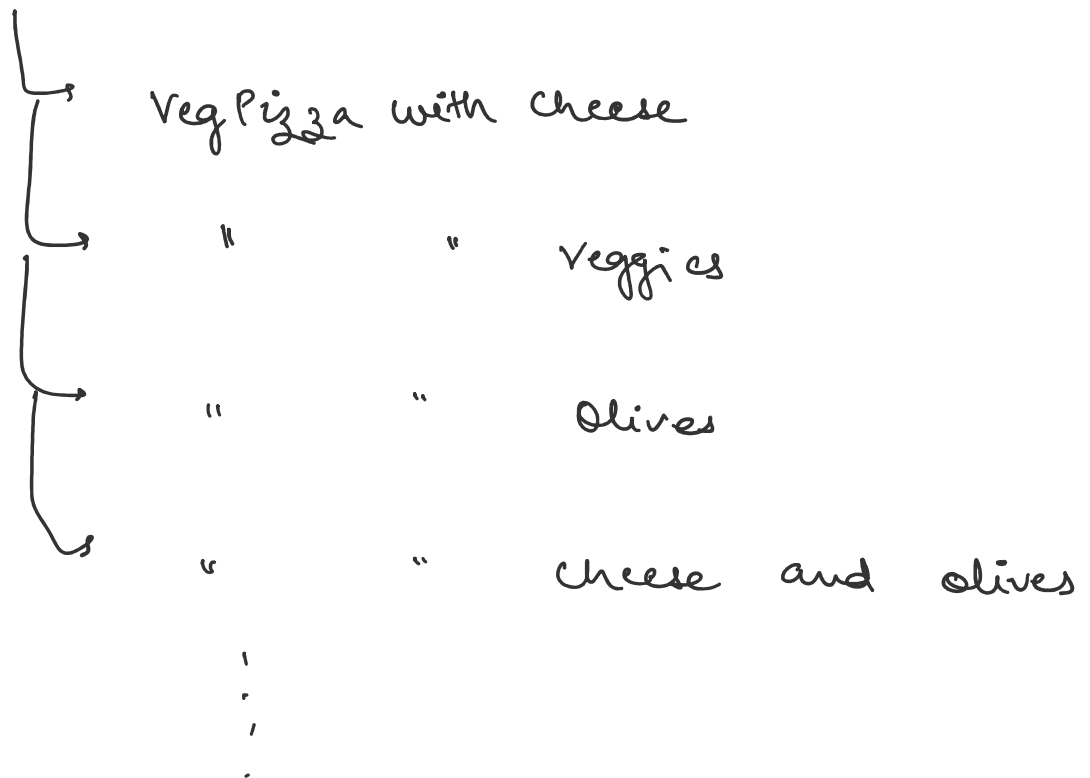
Pizza



Extra Toppings —

- Cheese
 - Veggies
 - Chicken
 - Olive
- 

Veg Pizza → Interface



Too many classes



- Difficult to maintain
- Duplicated code
- Not properly flexible



Class Explosion problem

Solⁿ -

use composition to create pizza at runtime.

✓ req Pizza



name

cost = 100



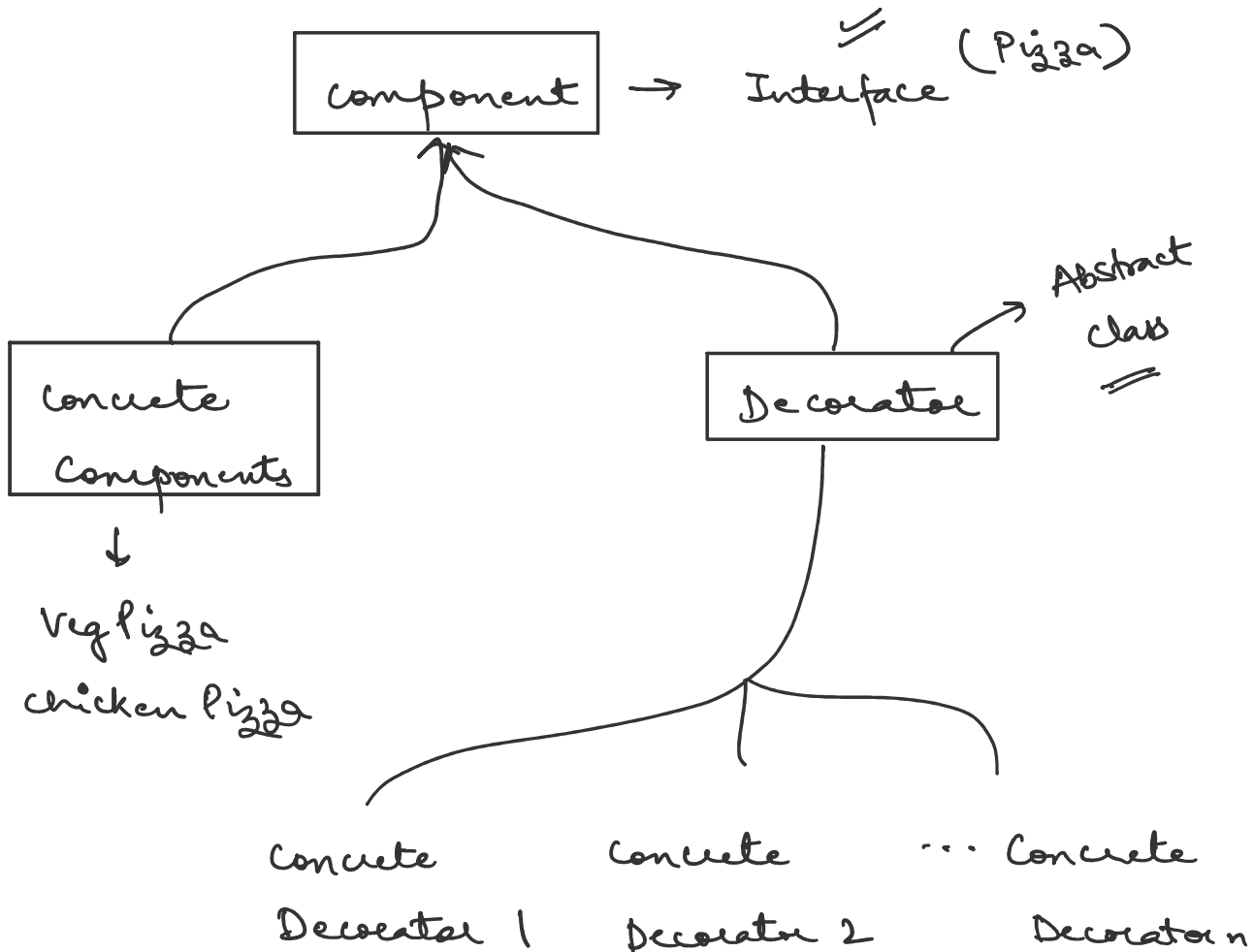
✓ Cheese Decorator → + 30



veggies Decorator → + 20

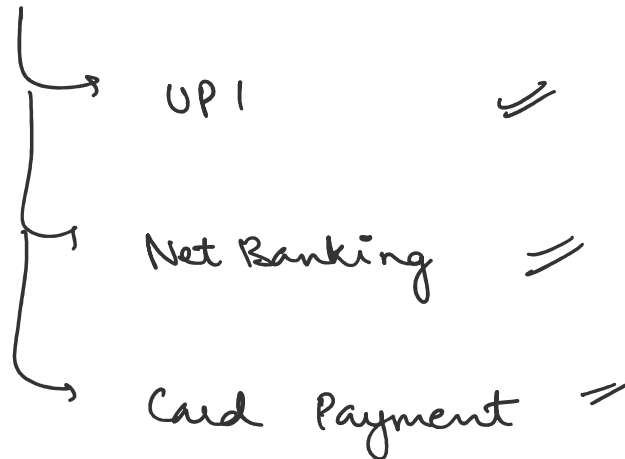


✓ Olives Decorator → + 25



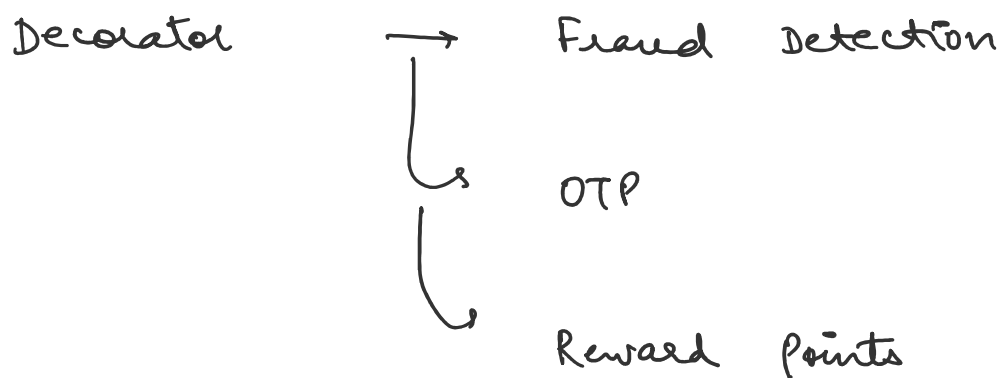
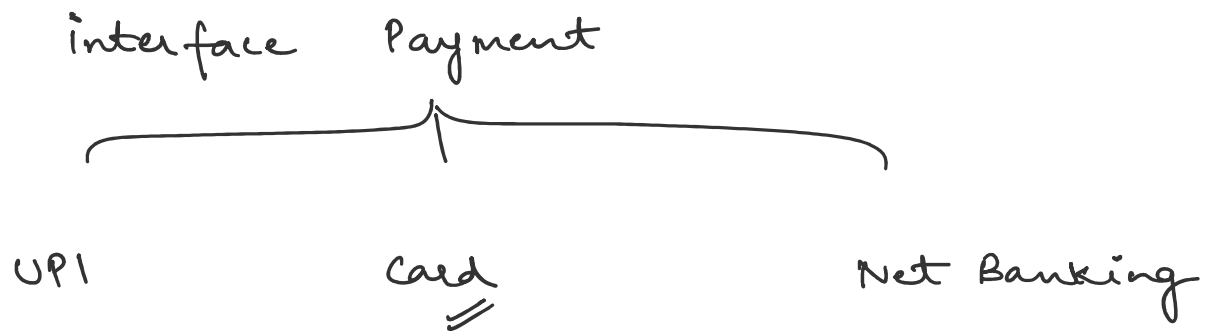
Another example → Payment Gateway

Payment Gateway

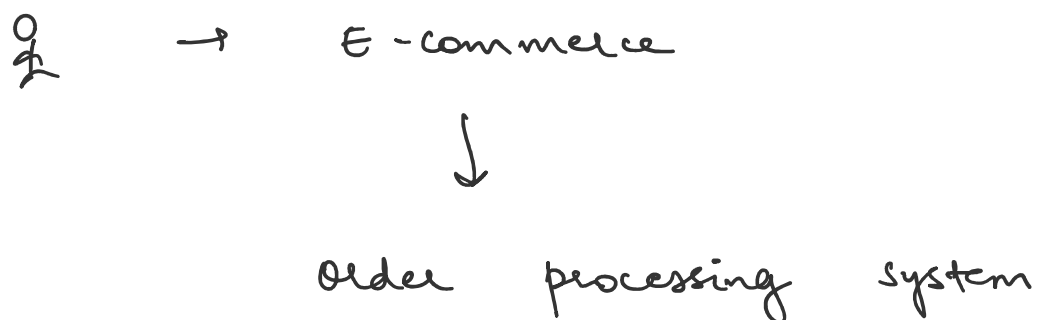


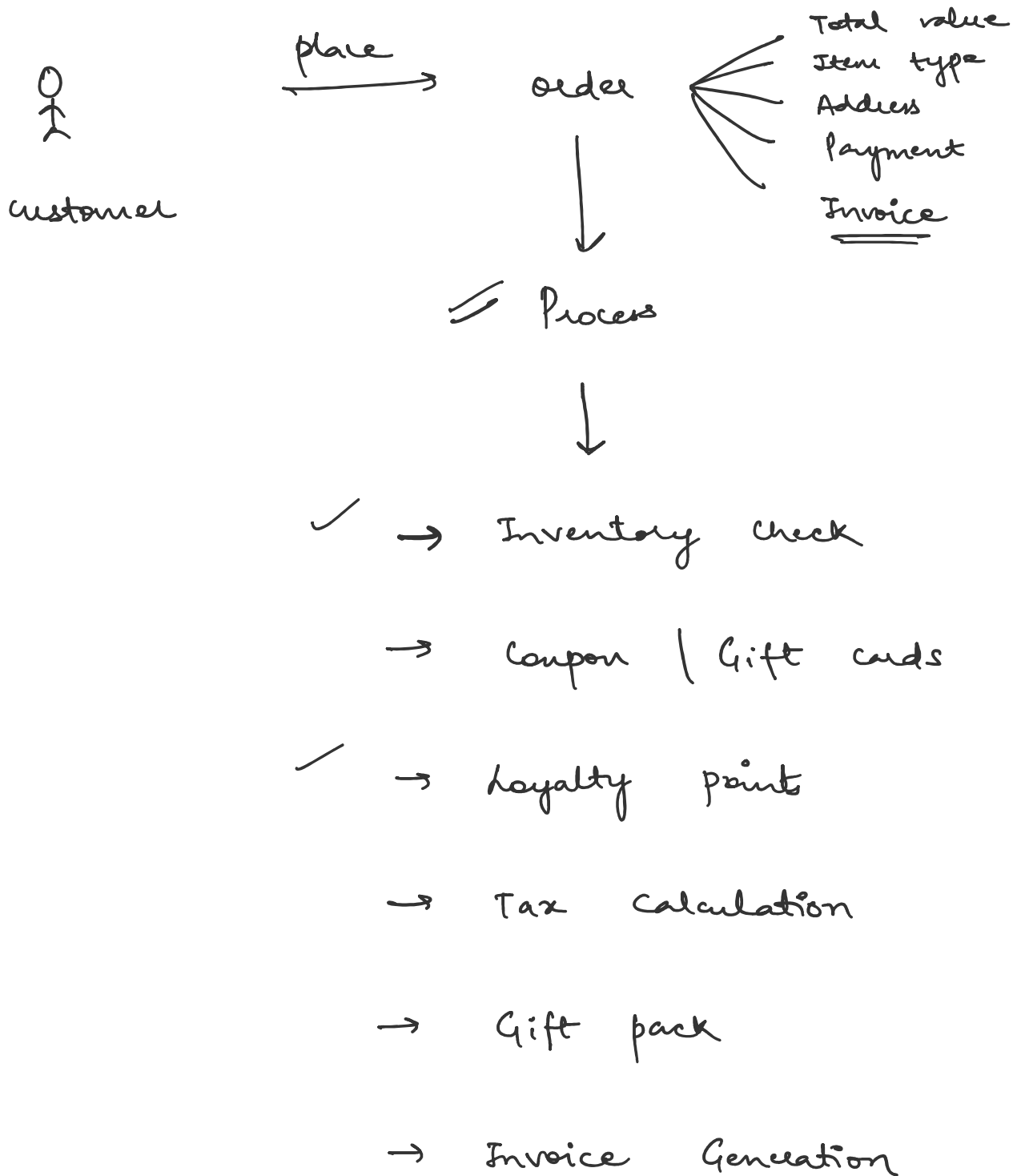
Additional Requirements -

- OTP verification
- Cashbacks
- Fraud Detections
- Reward points



Another example — order processing system





Notes :

Decorator Pattern

The **Decorator Pattern** is a **Structural Design Pattern** that allows us to **add new functionality to an object at runtime without modifying its existing code**.

Instead of creating many subclasses for every new feature, we **wrap the original object inside another object (called a decorator)** that adds additional behavior.

The original object remains unchanged, while the decorator extends its functionality.

This pattern follows the **Open/Closed Principle**:

- Open for extension
- Closed for modification

Instead of using inheritance, the Decorator Pattern uses **composition (object wrapping)** to achieve flexibility.

Why do we need the Decorator Pattern?

Suppose you own a pizza shop. Initially, your shop only sells two types of pizzas:

Pizza

```
|  
+---- VegPizza  
|  
+---- ChickenPizza
```

Later, customers start requesting extra toppings:

- Extra Cheese
- Extra Veggies
- Extra Chicken
- Olives

If you use inheritance, you'll need a new class for every combination.

```
VegPizza  
|  
+---- VegPizzaWithCheese  
|  
+---- VegPizzaWithVeggies  
|  
+---- VegPizzaWithOlives  
|  
+---- VegPizzaWithCheeseAndVeggies  
|  
+---- VegPizzaWithCheeseAndOlives  
|  
+---- VegPizzaWithCheeseVeggiesOlives
```

The same problem exists for ChickenPizza.

As more toppings are introduced, the number of classes keeps increasing.

This leads to:

- Too many classes
- Difficult maintenance
- Code duplication
- Poor flexibility

This problem is known as the **Class Explosion Problem**.

Solution

Instead of creating a new class for every topping combination, we decorate the pizza at runtime.

VegPizza



CheeseDecorator



VeggiesDecorator



OlivesDecorator

Each decorator adds only one topping.

Want extra cheese? Wrap the pizza with CheeseDecorator.

Want cheese and veggies? Wrap it with CheeseDecorator and then VeggiesDecorator.

Want cheese, veggies, and olives? Wrap all three decorators.

The original VegPizza class never changes. Features can be added,

removed, or reordered without changing existing classes. This makes the system much more flexible and easy to extend.

Think of it like **gift wrapping** :

- Base gift = Pizza.
- First wrapper = Cheese topping.
- Second wrapper = Chicken topping.

You still have the same gift, but with extra layers of functionality.

Another Example – Payment Processing

Suppose you're building a payment gateway.

Initially, the application supports:

- Credit Card
- UPI
- Net Banking

Later, business requirements include:

- Fraud Detection
- Audit Logging
- Cashback
- Reward Points
- OTP Verification

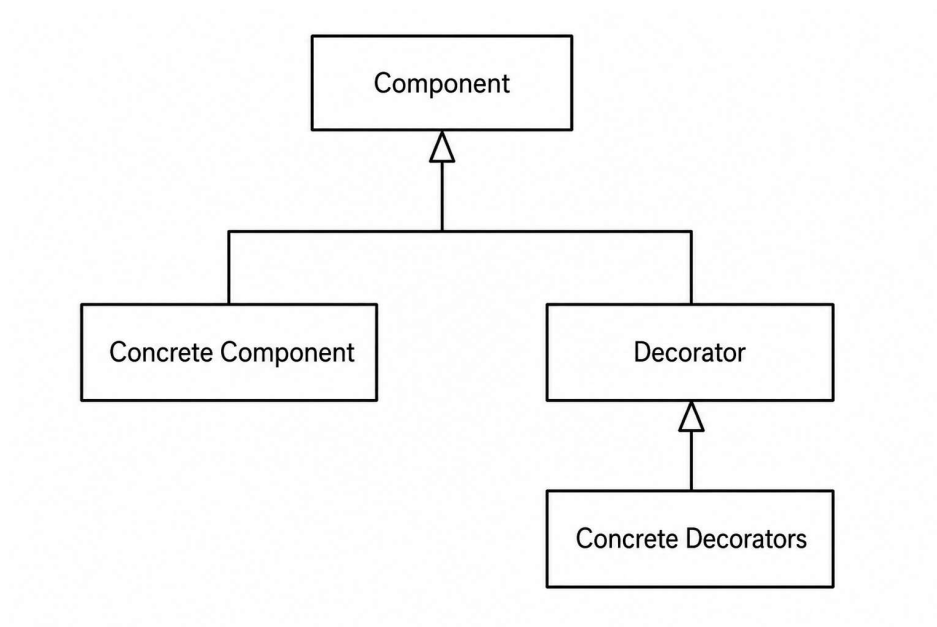
Instead of creating dozens of payment classes, decorate the payment object.

CardPayment
↓
FraudCheckDecorator
↓
AuditDecorator
↓
CashbackDecorator

Every feature is independent and can be added or removed without modifying the original payment class.

How the Decorator Pattern Works

Every Decorator Pattern has four main parts.



1. Component

Defines the common interface that both the original object and decorators will implement.

Example: Pizza

2. Concrete Component

The original object that contains the basic functionality.

Examples: VegPizza, ChickenPizza.

3. Decorator

An abstract class that also implements the same interface.

Instead of extending the concrete class, it **contains** an object of the same interface. This is where composition is used.

The decorator forwards all requests to the wrapped object.

4. Concrete Decorators

Each concrete decorator adds one additional responsibility.

Examples: CheeseDecorator, VeggiesDecorator, ChickenDecorator.

Each decorator performs its own work and then delegates the remaining work to the wrapped object.

Pizza example implementation

1. Define a Component Interface

```
public interface Pizza {  
    String getDescription();  
  
    double getCost();  
}
```

2. Create Concrete Components - Create simple classes like **VegPizza** and **ChickenPizza** that implement the **Pizza interface**. These represent the base pizzas — without any toppings. They provide the **default cost** and **description** for plain pizzas.

(i) VegPizza

```
public class VegPizza implements Pizza {  
    @Override  
    public String getDescription() {  
        return "Veg Pizza";  
    }  
  
    @Override  
    public double getCost() {  
        return 100.0;  
    }  
}
```

(ii) ChickenPizza

```
public class ChickenPizza implements Pizza {  
    @Override  
    public String getDescription() {
```

```
        return "Chicken Pizza";
    }

    @Override
    public double getCost() {
        return 150.0;
    }
}
```

3. Create abstract decorator - To add toppings without changing the base pizzas, we will use **composition over inheritance** — instead of extending a pizza, the decorator **wraps** it.

```
public abstract class PizzaDecorator implements Pizza {
    protected final Pizza pizza;

    public PizzaDecorator(Pizza pizza) {
        this.pizza = pizza;
    }

    @Override
    public String getDescription() {
        return pizza.getDescription();
    }

    @Override
    public double getCost() {
        return pizza.getCost();
    }
}
```

4. Create Concrete Decorators - Create individual decorators and each

one adds some logic to the cost and description — for example, cheese adds ₹30 and appends ", Cheese" to the name. Each decorator adds one specific feature — following the **Single Responsibility Principle**.

We can mix and match decorators in any order, based on what the customer wants — we can just chain decorators at runtime.

(i) VeggiesDecorator

```
public class VeggiesDecorator extends PizzaDecorator {  
    public VeggiesDecorator(Pizza pizza) {  
        super(pizza);  
    }  
  
    @Override  
    public String getDescription() {  
        return super.getDescription() + ", Extra Veggies";  
    }  
  
    @Override  
    public double getCost() {  
        return super.getCost() + 20.0;  
    }  
}
```

(ii) CheeseDecorator

```
public class CheeseDecorator extends PizzaDecorator {  
    public CheeseDecorator(Pizza pizza) {  
        super(pizza);  
    }  
  
    @Override
```

```

    public String getDescription() {
        return super.getDescription() + ", Cheese";
    }

    @Override
    public double getCost() {
        return super.getCost() + 30.0;
    }
}

```

(iii) ChickenToppingDecorator

```

public class ChickenToppingDecorator extends PizzaDecorator {
    public ChickenToppingDecorator(Pizza pizza) {
        super(pizza);
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", Extra Chicken";
    }

    @Override
    public double getCost() {
        return super.getCost() + 40.0;
    }
}

```

5. Create client code – main method

```

public class Main {
    public static void main(String[] args) {
        // Without constructor chaining
    }
}

```

```

    Pizza veggiePizza = new VegPizza(); // Start with a veggie
    pizza
    veggiePizza = new CheeseDecorator(veggiePizza); // Add
    cheese topping
    veggiePizza = new VeggiesDecorator(veggiePizza); // Add
    extra veggies

    System.out.println("Order: " + veggiePizza.getDescription());
    // Output: Order: Veggie Pizza, Cheese, Extra Veggies
    System.out.println("Total Cost: ₹" + veggiePizza.getCost());
    // Output: Total Cost: ₹150.0

```

```

    // With constructor chaining
    Pizza chickenPizza = new ChickenToppingDecorator( //
    Add Extra Chicken
        new VeggiesDecorator( // Add Extra Veggies
            new CheeseDecorator( // Add Cheese
                new ChickenPizza() // Base: Chicken Pizza
            )
        )
    );

    System.out.println("Order: " +
    chickenPizza.getDescription()); // Output: Order: Chicken Pizza,
    Cheese, Extra Veggies, Extra Chicken
    System.out.println("Total Cost: ₹" + chickenPizza.getCost());
    // Output: Total Cost: ₹240.0
}
}

```

Another Example: Payment Gateway

Suppose you're building a **Payment Gateway** that supports multiple payment methods like **UPI, Card, and Net Banking**.

Initially, each payment method only processes the payment. Later, the business introduces additional requirements such as:

- Fraud Detection
- OTP Verification
- Cashback
- Audit Logging
- Reward Points

If inheritance is used, you'll end up creating many classes like:

- CardPaymentWithFraudCheck
- CardPaymentWithOTP
- CardPaymentWithFraudAndOTP
- UPIPaymentWithCashback
- NetBankingPaymentWithAudit
- ...

As more features are added, the number of classes grows rapidly, making the system difficult to maintain.

Instead, we use the **Decorator Pattern**, where each feature is implemented as a separate decorator and can be added dynamically at runtime.

Step 1: Create Component Interface

```
public interface Payment {  
    void pay(double amount);  
}
```

Step 2: Create Concrete Components

CardPayment

```
public class CardPayment implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using Card");  
    }  
}
```

UPIPayment

```
public class UPIPayment implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using UPI");  
    }  
}
```

NetBankingPayment

```
public class NetBankingPayment implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using Net  
Banking");  
    }  
}
```

Step 3: Create Abstract Decorator

```
public abstract class PaymentDecorator implements Payment {
    protected final Payment payment;

    public PaymentDecorator(Payment payment) {
        this.payment = payment;
    }

    @Override
    public void pay(double amount) {
        payment.pay(amount);
    }
}
```

Step 4: Create Concrete Decorators

FraudCheckDecorator

```
public class FraudCheckDecorator extends PaymentDecorator {
    public FraudCheckDecorator(Payment payment) {
        super(payment);
    }

    @Override
    public void pay(double amount) {
        System.out.println("Checking for fraudulent transaction...");
        super.pay(amount);
        System.out.println("Fraud check passed.");
    }
}
```

OTPVerificationDecorator

```
public class OTPVerificationDecorator extends
PaymentDecorator {
    public OTPVerificationDecorator(Payment payment) {
        super(payment);
    }

    @Override
    public void pay(double amount) {
        System.out.println("Sending OTP...");
        System.out.println("OTP Verified Successfully.");
        super.pay(amount);
    }
}
```

CashbackDecorator

```
public class CashbackDecorator extends PaymentDecorator {
    public CashbackDecorator(Payment payment) {
        super(payment);
    }

    @Override
    public void pay(double amount) {
        super.pay(amount);
        double cashback = amount * 0.05;
        System.out.println("Cashback Earned : ₹" + cashback);
    }
}
```

AuditDecorator

```

import java.time.LocalDateTime;

public class AuditDecorator extends PaymentDecorator {
    public AuditDecorator(Payment payment) {
        super(payment);
    }

    @Override
    public void pay(double amount) {
        System.out.println("Audit Log:");
        System.out.println("Time : " + LocalDateTime.now());
        System.out.println("Amount : ₹" + amount);
        super.pay(amount);
        System.out.println("Transaction recorded successfully.");
    }
}

```

RewardPointsDecorator

```

public class RewardPointsDecorator extends PaymentDecorator {
    public RewardPointsDecorator(Payment payment) {
        super(payment);
    }

    @Override
    public void pay(double amount) {
        super.pay(amount);
        int points = (int) amount / 100;
        System.out.println("Reward Points Earned : " + points);
    }
}

```

Step 5: Client Code


```

public class Main {
    public static void main(String[] args) {
        System.out.println(":: Card Payment ::");

        Payment cardPayment = new RewardPointsDecorator(
            new CashbackDecorator(
                new AuditDecorator(
                    new OTPVerificationDecorator(
                        new FraudCheckDecorator(
                            new CardPayment()
                        )
                    )
                )
            )
        );

        cardPayment.pay(5000);
        System.out.println("-----");

        System.out.println(":: UPI Payment ::");

        Payment upiPayment = new CashbackDecorator(
            new AuditDecorator(
                new FraudCheckDecorator(
                    new UPIPayment()
                )
            )
        );

        upiPayment.pay(2500);
        System.out.println("-----");

        System.out.println(":: Net Banking Payment ::");

        Payment netBankingPayment = new
    
```

```

RewardPointsDecorator(
    new OTPVerificationDecorator(
        new AuditDecorator(
            new NetBankingPayment()
        )
    )
);

netBankingPayment.pay(10000);
}
}

/* *

```

Output:

:: Card Payment ::

Audit Log:

Time : 2026-07-05T14:46:55.466258300

Amount : ₹5000.0

Sending OTP...

OTP Verified Successfully.

Checking for fraudulent transaction...

Paid ₹5000.0 using Card

Fraud check passed.

Transaction recorded successfully.

Cashback Earned : ₹250.0

Reward Points Earned : 50

:: UPI Payment ::

Audit Log:

Time : 2026-07-05T14:46:55.481860900

Amount : ₹2500.0

Checking for fraudulent transaction...

Paid ₹2500.0 using UPI

Fraud check passed.

Transaction recorded successfully.

Cashback Earned : ₹125.0

:: Net Banking Payment ::
Sending OTP...
OTP Verified Successfully.
Audit Log:
Time : 2026-07-05T14:46:55.484955400
Amount : ₹10000.0
Paid ₹10000.0 using Net Banking
Transaction recorded successfully.
Reward Points Earned : 100

* */

Another Example : Order Processing System

Suppose you're building an **Order Processing System** for an e-commerce application.

Whenever a customer places an order, the system first processes the order. Later, the business introduces additional processing steps such as:

- Inventory Check
- Coupon Validation
- Loyalty Points
- Tax Calculation
- Invoice Generation

If inheritance is used, you'll end up creating many classes like:

- OrderProcessorWithInventory
- OrderProcessorWithCoupon
- OrderProcessorWithTax
- OrderProcessorWithInventoryAndCoupon

- OrderProcessorWithCouponAndInvoice
- ...

As more features are added, the number of classes grows rapidly, making the system difficult to maintain.

Instead, we use the **Decorator Pattern**, where each processing step is implemented as a separate decorator and added dynamically at runtime.

1. Create Order class

```
public class Order {  
    private String orderId;  
    private double amount;  
  
    public Order(String orderId, double amount) {  
        this.orderId = orderId;  
        this.amount = amount;  
    }  
  
    public String getOrderId() {  
        return orderId;  
    }  
  
    public double getAmount() {  
        return amount;  
    }  
}
```

2. Create a Component Interface

```
public interface OrderProcessor {  
    void process(Order order);  
}
```

3. Create Concrete Components - **BasicOrderProcessor** and **PriorityOrderProcessor**

- BasicOrderProcessor → Normal customers
- PriorityOrderProcessor → Premium/VIP customers

Both implement the same OrderProcessor interface, and decorators (Inventory, Coupon, Tax, Invoice, etc.) can be applied to either.

BasicOrderProcessor

```
public class BasicOrderProcessor implements OrderProcessor {  
    @Override  
    public void process(Order order) {  
        System.out.println("Processing Standard Order");  
        System.out.println("Order ID : " + order.getId());  
        System.out.println("Order Amount : ₹" + order.getAmount());  
    }  
}
```

PriorityOrderProcessor

```
public class PriorityOrderProcessor implements OrderProcessor {  
    @Override  
    public void process(Order order) {  
        System.out.println("Processing Priority/VIP Order");  
        System.out.println("Assigning dedicated processing
```

```

queue...");
    System.out.println("Order ID : " + order.getId());
    System.out.println("Order Amount : ₹" + order.getAmount());
}
}

```

4. Create abstract decorator

```

public abstract class OrderProcessorDecorator implements
OrderProcessor {
    protected final OrderProcessor processor;

    public OrderProcessorDecorator(OrderProcessor processor) {
        this.processor = processor;
    }

    @Override
    public void process(Order order) {
        processor.process(order);
    }
}

```

5. Create Concrete Decorators

(i) InventoryDecorator

You **must verify inventory before processing the order**. If inventory isn't available, you might not even proceed.

Check Inventory



Process Order

```

public class InventoryDecorator extends
OrderProcessorDecorator {
    public InventoryDecorator(OrderProcessor processor) {
        super(processor);
    }

    @Override
    public void process(Order order) {
        System.out.println("Checking inventory.");
        System.out.println("Inventory available.");
        super.process(order);
    }
}

```

(ii) CouponDecorator

Coupons are validated before placing the order.

Validate Coupon



Process Order

```

public class CouponDecorator extends OrderProcessorDecorator
{
    public CouponDecorator(OrderProcessor processor) {
        super(processor);
    }

    @Override
    public void process(Order order) {
        System.out.println("Validating coupon.");
        System.out.println("Coupon applied successfully.");
    }
}

```

```
        super.process(order);
    }
}
```

(iii) TaxDecorator

Tax is being calculated before the order is processed.

Calculate Tax



Process Order

```
public class TaxDecorator extends OrderProcessorDecorator {
    public TaxDecorator(OrderProcessor processor) {
        super(processor);
    }
}
```

```
@Override
public void process(Order order) {
    double tax = order.getAmount() * 0.18;
    System.out.println("Tax Calculated : ₹" + tax);
    super.process(order);
}
}
```

(iv) LoyaltyPointsDecorator

Customers earn loyalty points **after the order has been successfully processed.**

Think of Amazon or Flipkart:

You place order
↓
Payment succeeds
↓
Order confirmed
↓
Reward points credited

You wouldn't credit points before knowing whether the order actually succeeded.

Process Order
↓
Credit Reward Points

```
public class LoyaltyPointsDecorator extends
OrderProcessorDecorator {
    public LoyaltyPointsDecorator(OrderProcessor processor) {
        super(processor);
    }

    @Override
    public void process(Order order) {
        super.process(order);
        int points = (int) order.getAmount() / 100;
        System.out.println("Reward Points Earned : " + points);
    }
}
```

(v) InvoiceDecorator

Invoices are generated after the order has been accepted or

completed.

Process Order



Generate Invoice

```
public class InvoiceDecorator extends OrderProcessorDecorator {  
    public InvoiceDecorator(OrderProcessor processor) {  
        super(processor);  
    }  
}
```

```
@Override  
public void process(Order order) {  
    super.process(order);  
    System.out.println("Generating Invoice...");  
    System.out.println("Invoice Generated Successfully.");  
}  
}
```

6. Create client code – main method

```
public class Main {  
    public static void main(String[] args) {  
        Order order1 = new Order("ORD-101", 5000);  
        System.out.println(":: Basic Order ::");  
  
        OrderProcessor basic = new InvoiceDecorator(  
            new LoyaltyPointsDecorator(  
                new TaxDecorator(  
                    new CouponDecorator(  
                        new InventoryDecorator(  
                            new BasicOrderProcessor()  
                        )  
                    )  
                )  
            )  
        )  
    }  
}
```

```

        )
    )
);

basic.process(order1);
System.out.println("-----");

Order order2 = new Order("ORD-102", 7500);
System.out.println(":: VIP Order ::");

OrderProcessor vip = new InvoiceDecorator(
    new LoyaltyPointsDecorator(
        new TaxDecorator(
            new CouponDecorator(
                new InventoryDecorator(
                    new PriorityOrderProcessor()
                )
            )
        )
    )
);

vip.process(order2);
}
}

/* *

```

Output :

```

:: Basic Order ::
Tax Calculated : ₹900.0
Validating coupon.
Coupon applied successfully.
Checking inventory.
Inventory available.
Processing Standard Order

```

Order ID : ORD-101
Order Amount : ₹5000.0
Reward Points Earned : 50
Generating Invoice.
Invoice Generated Successfully.

:: VIP Order ::
Tax Calculated : ₹1350.0
Validating coupon.
Coupon applied successfully.
Checking inventory.
Inventory available.
Processing Priority/VIP Order
Assigning dedicated processing queue...
Order ID : ORD-102
Order Amount : ₹7500.0
Reward Points Earned : 75
Generating Invoice.
Invoice Generated Successfully.

* */

Feature	Decorator Pattern	Adapter Pattern
Main Purpose	Add new behavior or responsibilities dynamically at runtime	Convert one interface into another (i.e., interface translation)
Pattern Type	Structural pattern (adds behavior)	Structural pattern (translates interface)
Client View	Client sees the same interface , but with enhanced behavior	Client can use incompatible interfaces as if they're compatible

Uses Composition?	☒ Yes (wraps the object)	☒ Yes (wraps the object)
Method Chaining?	☒ Often used for dynamic layering	☒ Not typical; only adapts interface
Common Real-Life Example	Toppings on a pizza or adding filters to a stream	Phone charger adapter or legacy API to new system